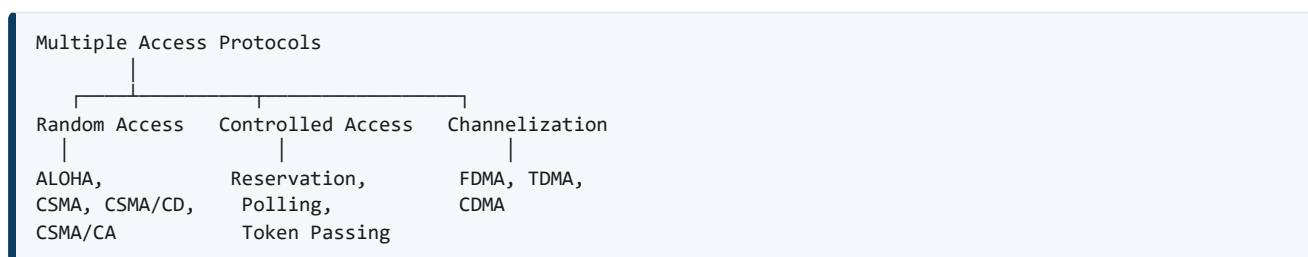


Multiple Access Protocols, Connecting Devices & Spanning Tree

PART 1 — Multiple Access Protocols

1. Why Multiple Access Protocols Exist

If a dedicated link exists between exactly two stations, plain data-link control is sufficient (see the Flow Control & ARQ topic). But on a **shared medium** — where multiple stations can all attempt to transmit at once — something has to decide **who gets to talk when**, or transmissions will collide and garble each other. Multiple access protocols solve exactly this.



- **Random access** — no station has priority over another; any station may transmit whenever it wants, based on the medium's state. No fixed schedule, no fixed order.
- **Controlled access** — stations coordinate who transmits next via reservation, polling (a controller asks each station in turn), or token passing (only whoever holds the token may transmit — as seen in ring topology).
- **Channelization** — the available bandwidth is statically divided (by frequency, time, or code) among stations, so each effectively has its own private sub-channel (this is really multiplexing, applied to the access problem).

This topic focuses on **random access**, since it's the richest and most commonly asked about.

2. ALOHA

2.1 Pure ALOHA

The simplest possible scheme: a station transmits **whenever it has data**, with no regard for what anyone else is doing.

- If no acknowledgment arrives within an expected window, the station assumes a **collision** occurred, waits a **random back-off time (T_b)**, and retransmits.
- Different stations choosing different random back-off times is precisely what reduces the *probability* of a repeated collision on retry.

Vulnerable period: since any station can start transmitting at any instant, a collision can happen if another station starts transmitting anywhere within **one full frame-time before or after** a given station's transmission — a vulnerable window of **$2 \times$ frame time**.

2.2 Slotted ALOHA

Improves on pure ALOHA by dividing time into discrete **slots**, and only permitting transmission to **begin at the start of a slot**.

- If a station's data becomes ready mid-slot, it must **wait for the next slot boundary**.
- This halves the vulnerable period to just **$1 \times$ frame time** (a collision can now only occur if two stations choose the *same* slot, not any overlapping timing at all) — significantly reducing collision probability compared to pure ALOHA.

Pure ALOHA: stations transmit any time $\rightarrow$ wide collision window
 Slotted ALOHA: |--slot--|--slot--|--slot--| $\rightarrow$ transmission only at slot boundaries

3. CSMA (Carrier Sense Multiple Access)

Core idea: "Sense before transmit" (a.k.a. "listen before talk") — a station checks whether the medium is idle **before** attempting to transmit, rather than transmitting blindly like ALOHA does.

Why collisions can still happen despite sensing: propagation delay. If station A senses the channel idle and starts transmitting, that signal takes non-zero time to physically reach station B. If B senses the channel *during that propagation window* — before A's signal has actually arrived — B will *also* find it idle and may also start transmitting, causing a collision anyway.

CSMA persistence strategies — what to do if the channel is found busy

Strategy	Behavior when busy	Trade-off
1-persistent	Continuously sense the channel; transmit immediately (with probability 1) the instant it becomes idle	Simple, but if multiple stations were all waiting for the same busy channel to free up, they'll likely all transmit the instant it clears → high collision chance right at that moment
Non-persistent	Wait a random amount of time, then sense again (not continuously)	Reduces the "everyone pounces at once" problem, but wastes some channel idle time while stations are randomly waiting instead of sensing
p-persistent	On a slotted channel, if idle, transmit with probability p ; with probability 1-p , wait for the next slot and repeat	Tunable middle ground between the aggressiveness of 1-persistent and the conservatism of non-persistent

1-persistent: sense—sense—sense—[idle]→ transmit immediately
Non-persistent: sense→[busy]→ wait(random) → sense→[idle]→ transmit

4. CSMA/CD (Collision Detection)

Adds **collision detection** on top of CSMA: while transmitting, a station **keeps listening** to the channel. If it detects that the signal on the wire doesn't match what it's sending (i.e., a collision occurred), it **immediately aborts** transmission — rather than wastefully continuing to send a frame that's already been garbled.

- After aborting, a **jam signal** is sent to ensure all stations are aware a collision happened, then each colliding station waits a random back-off time before retrying (similar spirit to ALOHA's back-off, but triggered by *detected* collision rather than a missing ACK).
- **This is the classic access method for traditional (shared-medium) Ethernet.**

Why CSMA/CD works well on wired Ethernet but not wireless: detecting a collision requires being able to simultaneously transmit and listen, and reliably notice that what you're sending differs from what's on the wire — feasible on a wired medium where signal strength is fairly uniform, but **not reliable on wireless**, where a station's own transmission can drown out its ability to hear a weaker colliding signal from far away (this is exactly the motivation for CSMA/CA below).

5. CSMA/CA (Collision Avoidance)

Since collisions can't be reliably *detected* on wireless media, CSMA/CA instead focuses on **avoiding** them proactively, using three main mechanisms:

1. **Interframe Space (IFS)** — a station senses the channel idle, but still waits a short additional fixed interval before transmitting, giving any higher-priority or already-queued transmission a chance to start first.
2. **Contention window** — a random back-off mechanism, similar in spirit to ALOHA/CSMA back-off, applied even *before* the first transmission attempt (not just after a collision).
3. **RTS/CTS handshake** (Request to Send / Clear to Send) — before sending actual data, the sender sends a short RTS frame; the receiver replies with a CTS frame. Both frames announce the intended transmission duration, so **other nearby stations that hear either frame know to stay silent** for that duration — directly addressing the "hidden terminal problem" (two wireless stations that can each reach a common receiver, but can't hear each other directly, and so wouldn't otherwise know the medium is about to be busy).
- **This is the access method used by Wi-Fi (802.11).**

Interview soundbite: "CSMA/CD detects collisions *after* they start and reacts fast, which works because a wired station can reliably tell its own strong signal apart from a colliding one. CSMA/CA can't rely on that on wireless, so it shifts the whole strategy to *avoidance* — interframe spacing, random back-off before even trying, and RTS/CTS reservation — accepting more overhead per transmission in exchange for not needing reliable in-flight collision detection at all."

PART 2 — Connecting Devices

6. The Hierarchy, by OSI Layer

Device	OSI Layer	Behavior
Repeater	Physical (1)	Regenerates/amplifies a weakening signal to extend cable range — no awareness of addresses at all
Hub	Physical (1)	A multiport repeater — broadcasts whatever arrives on one port to every other active port , with zero intelligence about who actually needs it
Bridge / Switch	Data Link (2)	Forwards frames intelligently based on MAC addresses , rather than blindly broadcasting to every port
Router	Network (3)	Forwards packets based on IP addresses , connecting distinct networks together

7. Switch — MAC Address Learning

A switch builds and maintains a **MAC learning table** (also called a forwarding table): `{Port Number → Source MAC address seen on that port}`.

Step-by-step, when a switch receives a frame:

1. **Learn:** look at the frame's **source MAC address** — if it's not already in the table, add an entry mapping that MAC to the port it arrived on.
2. **Forward:** look at the frame's **destination MAC address** in the table.
 - If found → forward the frame **only to that specific port** (unicast — efficient, no wasted bandwidth on other ports).
 - If not found (unknown destination) → **broadcast** the frame to all other active ports (flood), since the switch doesn't yet know where that destination actually is.

Learning table (initially empty):

Port	Source MAC
-----	-----

PC1 (00:11:11:11:11:11) sends a frame to PC4 (00:44:44:44:44:44)

Switch learns: Port 1 → 00:11:11:11:11:11

Switch doesn't know PC4's port yet → floods to ports 2, 3, 4

PC4 replies → Switch learns: Port 4 → 00:44:44:44:44:44

Now: any future frame from PC1 to PC4 is forwarded DIRECTLY to port 4 only
(ports 2, 3 no longer see this traffic at all)

Why only the first packet is flooded: once the switch has learned *both* the source and (from a reply) the destination MAC's port, subsequent frames between that pair are sent as a direct, unicast forwarding decision — dramatically reducing unnecessary traffic on unrelated ports compared to a hub, which would broadcast every single frame, every time, forever.

Interview soundbite: "A hub is a dumb, physical-layer broadcaster with no memory. A switch is a bridge with multiple ports and a learning table — it *watches* traffic to build up knowledge of where each MAC address actually lives, and only floods when it genuinely doesn't know yet. This single distinction (learning + selective forwarding vs. blind broadcasting) is why switches scale to far larger, higher-traffic networks than hubs ever could."

PART 3 — Spanning Tree Protocol (STP)

8. The Problem STP Solves

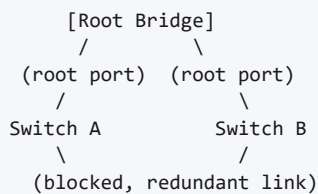
Redundant physical links between switches are good for **fault tolerance** (recall the mesh-topology trade-off from Network Topologies) — but at the Data Link layer, redundant paths between switches create **loops**, and loops are catastrophic here: a broadcast frame can circulate endlessly, being duplicated and re-forwarded forever — a **broadcast storm** that can bring the whole network down.

STP's job: keep the physical redundancy (for fail-over), but logically **block** enough links so the *active* topology, from a forwarding perspective, is always **loop-free** — literally a spanning tree of the physical network graph.

9. How STP Elects a Root Bridge and Builds the Tree

1. **Bridge ID (BID)** = Priority + MAC address. Every switch initially assumes *it* is the root and advertises its own BID via **BPDUs (Bridge Protocol Data Units)** exchanged with neighbors.

2. Each switch compares every BPDUs it receives against its own current belief: a **lower BID wins** (lower priority value first; MAC address as the tiebreaker). Switches update their view of "who's the root" accordingly, and this converges to agreement on a single **root bridge** — the switch with the globally lowest BID.
3. **Root Port** — every *non-root* switch determines the port offering the **shortest (lowest-cost) path** to the root bridge, based on a cost metric derived from link speed (faster links = lower cost). That port is designated the switch's root port.
4. **Loop elimination** — every other **redundant** path (a "non-root port" on some switch) is identified and put into a **blocking state** — physically still connected, but not used to forward regular traffic, unless the active path fails.
5. **Reacting to topology changes** — if an active link/switch fails, affected switches recalculate the shortest path to the root and can bring a previously-blocked port back into forwarding state to restore connectivity.



10. Advantages / Disadvantages

- **✓ Loop prevention** without sacrificing physical redundancy — prevents broadcast storms while still allowing failover paths to exist.
- **✓ Self-configuring** — no manual per-link configuration needed; root election and path calculation happen automatically.
- **✓ Scalable** to large, complex switch topologies.
- **✗ Convergence time** — recalculating the tree after a topology change isn't instantaneous; traffic can be delayed/disrupted during convergence, especially in larger networks.
- **✗ Wasted capacity** — blocked links sit unused during normal operation, meaning available bandwidth (and redundant hardware) isn't actually being used most of the time.

11. STP Variants

Version	Standard	Key improvement
STP	IEEE 802.1D	The original — correct, but slow convergence
RSTP (Rapid STP)	IEEE 802.1w	Much faster convergence, by reducing the number of states a port must pass through during a topology change
MSTP (Multiple STP)	IEEE 802.1s	Runs multiple independent spanning-tree instances, one per VLAN (or group of VLANs) — improves resource utilization since different VLANs can use different links as their "active" path, rather than all being bottlenecked by one single tree

Interview soundbite: "Plain STP's biggest practical weakness is that blocked links sit completely idle — even across many VLANs which could each benefit from a different path. RSTP addresses *speed* of recovery; MSTP addresses *utilization*, by giving each VLAN its own spanning tree instead of forcing every VLAN onto the exact same one."

Interview Questions With Answers

Q1. Why does slotted ALOHA have half the vulnerable period of pure ALOHA?

Answer: In pure ALOHA, a station can start transmitting at any arbitrary instant, so a collision occurs if another station's transmission overlaps at all — which can happen if that other station starts anywhere within one full frame-time before or after, giving a vulnerable window of $2 \times$ frame time. In slotted ALOHA, transmissions can only begin at fixed slot boundaries, so a collision can only occur if two stations choose to transmit in the exact same slot — narrowing the vulnerable window to just $1 \times$ frame time.

Q2. Why can collisions still occur in CSMA even though stations sense the channel before transmitting?

Answer: Because of propagation delay — sensing the channel only reflects its state *at the sensing station*, at that instant. If station A starts transmitting, its signal takes finite time to physically reach station B; if B happens to sense the channel during that propagation window, before A's signal has

arrived, B will find the channel idle (from its own vantage point) and may also transmit, causing a collision despite both stations having "correctly" sensed before transmitting.

Q3. Compare 1-persistent and non-persistent CSMA — what specific problem does each expose?

Answer: 1-persistent CSMA continuously senses a busy channel and transmits immediately (with certainty) the instant it becomes idle — this maximizes channel utilization when only one station is waiting, but if *multiple* stations were all waiting for the same busy channel, they'll all attempt to transmit at the exact same instant it clears, causing a near-guaranteed collision right at that moment. Non-persistent CSMA avoids this by waiting a random interval before re-sensing rather than sensing continuously, spreading out when different waiting stations attempt to transmit — but this can waste channel idle time, since a station might still be in its random wait period even though the channel has been free for a while.

Q4. Why doesn't CSMA/CD work well for wireless networks, motivating CSMA/CA instead?

Answer: CSMA/CD requires a station to detect a collision *while* it's transmitting, by comparing what it's sending against what's actually on the medium. On a wired medium, signal levels are fairly consistent, making this comparison reliable. On wireless, a station's own transmission is typically much stronger (from its own antenna) than a weaker colliding signal arriving from a distant station, so the station often can't reliably detect that a collision occurred at all — sometimes it can't even "hear" while it's "talking" on the same channel. CSMA/CA sidesteps this entirely by focusing on *avoiding* collisions proactively (interframe spacing, contention windows, RTS/CTS) rather than depending on detecting them once they start.

Q5. What is the hidden terminal problem, and how does RTS/CTS solve it?

Answer: The hidden terminal problem occurs when two wireless stations can each communicate with a common receiver, but can't hear each other directly (e.g., they're on opposite sides of an access point, out of each other's radio range) — so neither knows when the other is about to transmit, leading to collisions at the shared receiver that neither sender can sense coming. RTS/CTS solves this because the receiver's CTS reply (announcing the intended transmission duration) is broadcast and can be heard by *both* stations, even though they can't hear each other's RTS directly — any station hearing either the RTS or the CTS learns to stay silent for the announced duration, preventing the collision regardless of which stations can directly hear each other.

Q6. How does a switch decide when to flood a frame to all ports versus forward it to just one?

Answer: A switch maintains a MAC learning table mapping MAC addresses to the port they were last seen on. When a frame arrives, it looks up the frame's *destination* MAC address in this table: if there's a matching entry, it forwards the frame only to that specific port (unicast). If there's no entry yet (the switch has never seen a frame *from* that destination MAC, so it doesn't know which port

leads to it), it floods the frame to every other active port, exactly like a hub would — but only until a reply from that destination lets it learn the correct port, after which it stops flooding for that destination.

Q7. Why is STP necessary even though redundant links between switches are a good thing for fault tolerance?

Answer: Redundant links create physical loops in the network graph, and Ethernet frames (especially broadcasts) have no inherent mechanism to prevent endlessly circulating around a loop — a single broadcast frame can be duplicated and re-forwarded forever around the loop, exponentially multiplying traffic into a broadcast storm that can overwhelm and effectively take down the network. STP resolves this tension by logically blocking just enough redundant links so the *active forwarding topology* is always loop-free, while keeping the physical redundant links present and ready to be activated automatically if the currently active path fails.

Q8. How is the root bridge elected in STP, and why does a lower Bridge ID win rather than a higher one?

Answer: Every switch has a Bridge ID composed of a configurable priority value and its MAC address; switches exchange BPDUs advertising their BID, and each switch updates its belief about who the root is whenever it hears a BID lower than the one it currently believes is the root, eventually converging on the single lowest BID in the network as the root bridge. The specific convention (lower wins) is essentially arbitrary — what matters is that it's a consistent, universally comparable tiebreaker (a strict total order over Bridge IDs) that guarantees every switch in the network converges on the *same* unique root, and using priority-then-MAC-address as that ordering lets network administrators deliberately influence which switch becomes root (by setting a lower priority on a specific well-connected switch) while MAC address guarantees a unique tiebreak even if priorities are equal.

Q9. Scenario: A company has multiple VLANs on one physical switch topology, and plain STP is blocking the same links regardless of which VLAN's traffic is being considered. What's the practical downside, and which STP variant addresses it?

Answer: The downside is inefficient use of available capacity — the blocked link(s) sit completely idle for *every* VLAN, even though a different VLAN's traffic pattern might have been perfectly well served by using that "blocked" link as its active path instead, while blocking a different, otherwise-idle link. MSTP (Multiple Spanning Tree Protocol) addresses this by running multiple independent spanning-tree instances, one per VLAN (or group of VLANs), so different VLANs can each have their own active/blocked link assignment tailored to their own traffic — collectively making far better use of the physical redundant links across the network as a whole, rather than forcing every VLAN through one single shared tree.